

RMIT

ISYS3476 (Postgraduate)

Managing Semi-structured and Unstructured Data Assignment 1 Part B: Search Engine Implementation

Assessment	Assignment 1 Part B
Type	Individual
Weighting	30%
Due Date	Sunday, 13 September 2026, 11:59 pm (Melbourne time)
Submission	Canvas ZIP
Late Penalty	10% of the total available marks per calendar day or part thereof. Submissions more than five calendar days late will not be marked unless an approved extension, equitable assessment arrangement, or special consideration applies.
Total Points	30 points

Starter code and final submission

Starter code: Download and unzip the provided starter ZIP from Canvas. Before starting the tasks, follow README.md in the starter folder to set up Python and install the required packages.

Final submission: Submit one completed assignment ZIP on Canvas. It must contain your code, runs, WRITEUP.pdf, and recorded demonstration together. The Canvas ZIP is the only source used for marking.

What to read. This Main Specification defines the tasks and submission requirements, and partB_rubric_PG.pdf defines the marking standards. If you would like a more detailed review of the Task 1 semantics taught in class, consult partB_appendix_PG.pdf for worked examples and edge-case illustrations.

Contents

1 Overview and What You Submit	3
1.1 Tasks and Points: 30 Total	3
1.2 Deliverables	3
1.3 Starter Contents	3
2 Quick Start	4
3 Task 1: Indexing and Structured Retrieval	4
3.1 Index Construction and Access	4

3.2	Structured Query Processing	6
3.3	Toy Corpus Examples and Expected Outputs	6
4	Task 2: Multi-Algorithm Ranking	7
4.1	Goal	7
4.2	Ranking Function	7
4.3	Dev-Set Pearson Evaluation	7
4.4	Toy Corpus Ranking Example	8
4.5	Write-up requirements for Task 2	8
5	Task 3: System Optimisation	8
5.1	Provided end-to-end runner	8
5.2	Non-Ranking Optimisation Requirement	9
5.3	Dev-Set MAP Evaluation	9
5.4	Write-up requirements for Task 3	9
6	One-Page Write-up	10
6.1	Generative AI Use Declaration	10
7	Recorded CLI Demonstration	10
8	Overall Code Quality Standards	11
9	Submission Rules and Checklist	11
9.1	Submission Layout	11
9.2	Repository Rules	12
9.3	Pre-submission checklist	12
10	Academic Integrity and Prohibited Practices	12
10.1	Generative AI Condition	12

1 Overview and What You Submit

Building upon your Part A preprocessing pipeline, you will implement a complete information retrieval system with indexing, query processing, ranking, and one system-level optimisation.

1.1 Tasks and Points: 30 Total

- **Task 1 (8 pts):** Complete the index builder and three structured query processors:
 - `index/builders.py`: `create_all_indexes(...)`,
 - `query_processing/boolean.py`: `process_boolean_query(...)`,
 - `query_processing/wildcard.py`: `process_wildcard_query(...)`,
 - `query_processing/proximity.py`: `process_proximity_query(...)`.
- **Task 2 (8 pts):** Implement `ranking/rankers.py`: `rank_documents(...)` and compare ranking methods using dev-set Pearson correlation.
- **Task 3 (9 pts):** Use the provided `system/search_system.py` pipeline to implement **one non-ranking** optimisation and report dev MAP comparisons.
- **Recorded CLI demonstration and technical explanation (3 pts):** Run the self-checker and one evaluation command live, then use a relevant submitted code block to explain one visible Pearson or MAP result.
- **Code quality (2 pts):** Assessed holistically across your submission code base.

1.2 Deliverables

Your submission ZIP must include:

- **Code** for Tasks 1–3 in the required repository layout (Section 9).
- **Runs:** `runs/run_no_optimisation.json` and `runs/run_default.json`. If you aim for the top reporting band, include extra optimisation run files with short descriptive names.
- **Write-up:** `WRITEUP.pdf`, with a one-page report using 11pt text. If you use Generative AI, include the declaration described in Section 6.1.
- **Recorded demonstration:** `DEMONSTRATION.mp4` or `DEMONSTRATION.mov`, following Section 7 and submitted in the same ZIP as all other assignment files.

1.3 Starter Contents

The starter provides:

- the end-to-end pipeline entrypoint `system/search_system.py`,
- dev evaluation scripts `metrics/eval_pear.py` and `metrics/eval_map.py`,
- dev data under `data/dev/` and `data/dev_ranking/`,
- the unified term-index construction in `index/builders.py` and access helpers in `index/access.py`.

Your completed index package must remain compatible with the provided construction and access helpers.

2 Quick Start

- i) Download and extract the starter ZIP.
- ii) In VS Code, select **File > Open Folder** and open the extracted folder itself:
`ir-ass1-partB-startcode-PG`.
- iii) Follow the guided setup in `README.md` to create a local `.venv`, select Python **3.10 to 3.14 inclusive**, install `requirements.txt`, and install the required NLTK resources.
- iv) When setup finishes, select **Terminal > New Terminal**. Confirm that the prompt begins with `(.venv)`. Run the commands below from this terminal while the extracted starter folder is open.

Having trouble setting up Python or VS Code?

See [Python Setup and Troubleshooting](#).

Having trouble importing Python functions between project folders?

See [Python Project Imports and Running Modules](#).

Typical commands (all using `python -m ...`):

```
# 1) Sanity import + CLI smoke test
python -m test_sanity.check_submission

# 2) Evaluate rankers: starter default initially points to keyword_match
python -m metrics.eval_pear keyword_match
python -m metrics.eval_pear default

# 3) Produce the Task 3 no-optimisation baseline run
python -m system.search_system data/dev/queries.json data/dev/documents.
      jsonl runs/run_no_optimisation.json no_optimisation

# 4) Produce your submitted default system run; initially matches
      no_optimisation
python -m system.search_system data/dev/queries.json data/dev/documents.
      jsonl runs/run_default.json default

# 5) Evaluate MAP over all runs in ./runs/
python -m metrics.eval_map
```

Recommended workflow: Run `python -m test_sanity.check_submission` after setup and after major changes, then complete steps (1)–(5) again before submission.

`check_submission.py` checks interfaces and a small set of sanity cases; passing it is required for runnability but does not guarantee full marks.

3 Task 1: Indexing and Structured Retrieval

3.1 Index Construction and Access

Index package. The starter already builds the unified term index. Complete **one** serialized index package (one file on disk) containing three sub-indexes:

1. **Unified term index** for Boolean lookup,
2. **Wildcard index** (character n -grams with \$ boundaries) for term expansion,
3. **Proximity index** for unigram positions.

The provided unified entries also store document frequency and per-document term frequency for ranking methods such as BM25 in Task 2. You may adapt your Part A Task 2 character n -gram and position-mapping ideas.

Index builder. Complete the provided builder without changing its public contract:

```
create_all_indexes(
    tokenized_docs: List[List[str]],
    index_path: str,
    doc_ids: Optional[List[int]] = None
) -> None
```

Inputs:

- **tokenized_docs:** **already-cleaned** token lists. Use them **as-is**, without normalization, stemming, or token edits. Task 3 preprocessing belongs outside this function.
- **doc_ids:** optional list of integer IDs aligned with **tokenized_docs**. If **None**, assume IDs $0..N-1$.

Index package requirements:

- Build **one file** that serializes the inverted index dictionary to **index_path**.
- Preserve the provided unified term-index construction and package schema.
- Retain every supplied document ID, including IDs whose token list is empty.
- Posting lists and position lists must be **deduplicated** and **sorted ascending**.
- Character n -grams for wildcard: up to length 3, using \$ boundaries; **do not index the unigram "\$**".

Provided access functions. The starter code provides the following access helpers. You do **not** need to implement these functions, but your index package must support them:

```
get_posting_list(term, index_path) -> List[int]
get_all_doc_ids(index_path) -> List[int]
find_wildcard_matches(ngram, index_path) -> List[str]
get_term_positions(term, doc_id, index_path) -> List[int]
```

Access behaviour:

- **get_posting_list(term, ...)** returns the list of document IDs containing the unigram token **term**.
- **get_all_doc_ids(...)** returns all document IDs supplied to **create_all_indexes(...)** in ascending order.
- **find_wildcard_matches(ngram, ...)** returns the lexicographically sorted list of **terms** that contain the character n -gram **ngram** (with \$ boundaries). Example tri-gram input: "\$c1" or "te\$". Input is **not** a full wildcard pattern.
- **get_term_positions(term, doc_id, ...)** returns the sorted list of 0-based token positions of **term** within the document.

3.2 Structured Query Processing

Query processors. Implement the following functions:

```
process_boolean_query(query: str, index_path: str) -> Set[int]
process_wildcard_query(pattern: str, index_path: str) -> Set[int]
process_proximity_query(query: str, index_path: str) -> Set[int]
```

The starter separates each processor into smaller stages and includes selected code adapted from the Week 04 workshop. The workshop-derived code covers only the subset completed there. You may modify the helper functions, but the required public function contracts must remain unchanged.

Boolean queries. Evaluate Boolean expressions with AND/OR/NOT and parentheses. Operator precedence is NOT > AND > OR.

Wildcard queries. Expand a wildcard pattern (containing *) using the wildcard sub-index, then return the union of posting lists of all matched terms.

- Patterns contain exactly one *. It may match any sequence of characters, including the empty sequence, and may appear at the start (*ing), end (climat*), or inside the word (learn*ing).
- Use the wildcard n -gram index (character n -grams with \$ boundaries) to generate candidate terms, then filter candidates to those matching the full pattern.

Proximity queries. Evaluate queries of the form X NEAR/ k Y, where each operand is a unigram term. Use the 0-based term positions from the Task 1 proximity index and return documents where there exists at least one occurrence of X and one occurrence of Y whose token distance is at most k (order does not matter). If X and Y are the same term, they must be matched by two distinct occurrences.

Efficiency Warning: Hidden tests include efficiency/stress cases; see the rubric for mark caps. Query processors should use the built index package rather than scanning all documents or rebuilding indexes during query processing.

3.3 Toy Corpus Examples and Expected Outputs

All examples below use the same tiny input.

Toy input.

```
doc_ids = [191, 279]
tokenized_docs = [
    ["climate", "change", "policy"],      # doc 191
    ["machine", "learning", "improves"]   # doc 279
]
```

Build and access.

```
path = "index_pkg.pkl"
create_all_indexes(tokenized_docs, path, doc_ids=doc_ids)
get_posting_list("climate", path) # -> [191]
find_wildcard_matches("$cl", path) # -> ["climate"]
get_term_positions("change", 191, path) # -> [1]
```

Structured queries.

```
process_boolean_query("climate AND NOT learning", path) # -> {191}
process_boolean_query("climate OR machine", path) # -> {191, 279}
process_wildcard_query("climat*", path) # -> {191}
process_wildcard_query("*ing", path) # -> {279}
```

```
process_wildcard_query("learn*ing", path) # -> {279}
process_proximity_query("climate NEAR/3 policy", path) # -> {191}
```

For optional extra guidance on Task 1, including worked examples and edge-case illustrations, see `partB_appendix_PG.pdf`.

4 Task 2: Multi-Algorithm Ranking

4.1 Goal

Implement a ranking function that scores and orders candidate documents. You will compare at least two methods on the development set and connect your selected final ranking method to `method="default"`. A simple matching-based baseline is provided, so you only need to implement **one additional method** of your own. You may adapt Part A Tasks 3–4, build a new ranker using TF-IDF, dense-vector ranking, or BM25, or combine complementary ranking signals.

4.2 Ranking Function

```
rank_documents(
    query_toks: List[str],
    candidate_docs: List[List[str]],
    doc_ids: List[int],
    inverted_index_path: str,
    method: str = "default"
) -> Tuple[List[int], List[float]]
```

Function behaviour:

- Return every candidate document in ranked order. `ranked_ids` must be a permutation of `doc_ids`, with no additions or omissions.
- **Deterministic tie-break:** order equal scores by ascending `doc_id`.
- **Do not** perform query expansion or data cleaning inside `rank_documents`.
- Define `method="default"` as your selected final ranking method with fixed hyperparameters. The autograder evaluates `method="default"`; if it still points to the starter baseline, the starter baseline is what will be graded.

Efficiency Warning: Hidden tests include efficiency/stress cases; see the rubric for mark caps.

4.3 Dev-Set Pearson Evaluation

Use the provided script:

```
python -m metrics.eval_pear <method>
```

This script evaluates the dedicated ranking collection in `data/dev_ranking/` using the fixed Task 2 tokenization, independently of your Part A preprocessing, and computes Pearson correlation between predicted scores and graded relevance. Dev and hidden queries differ, so the dev value is guidance rather than a guaranteed mark. Use `python -m metrics.eval_pear default` to check the method that will be graded.

4.4 Toy Corpus Ranking Example

```
ranked_ids, scores = rank_documents(
    query_toks=["climate", "change"],
    candidate_docs=tokenized_docs,    # toy corpus docs
    doc_ids=doc_ids,                  # [191, 279]
    inverted_index_path="index_pkg.pkl",
    method="bm25"
)

# Typical lexical behaviour on the toy corpus:
# ranked_ids -> [191, 279]    (since doc 191 contains both query terms)
```

4.5 Write-up requirements for Task 2

Method description. Describe each compared ranking method. State its scoring signals, assumptions, and key settings. If a method combines or modifies standard approaches, explain its components, how they are combined, and what differs from the standard versions.

Results. Include a compact table comparing at least `keyword_match` and `default`. For each included method, report its key setting and dev Pearson result. Any ranking evidence cited in the report must be reproducible from `ranking/rankers.py` using `python -m metrics.eval_pear`.

Analysis. Compare `keyword_match` with `default` and justify the final choice. Examine stability or sensitivity across concrete queries or settings. Explain how the methods' scoring principles and assumptions affected the document scores. Then explain the differences in Pearson correlation in terms of how it is calculated and what it measures about agreement with graded relevance. Show how the evidence informed a later setting or confirmed the final choice while weighing limitations and trade-offs.

Highest band. Evaluate one additional hyperparameter or ranking-setting variant beyond `default`, using concrete document or term evidence from the dev runs. Explain whether the evidence supports adopting or rejecting the setting. This requirement is for the final 1 mark of the Task 2 reporting component.

5 Task 3: System Optimisation

5.1 Provided end-to-end runner

The end-to-end search pipeline is provided. You run it as:

```
python -m system.search_system <queries_json> <documents_jsonl> <
    run_output_json> <method>
```

If Tasks 1–2 are implemented correctly, this pipeline should run with minimal additional effort.

Task 3 uses a noisy-text view of the Task 2 document collection and a separate query workload. Its MAP queries are natural-language or Boolean queries. Wildcard and proximity correctness are assessed in Task 1 rather than through MAP.

Output format (expected output schema):

```
[
  {"qid": "Q41", "doc_ids": [142, 87, 203, ...], "scores": [0.95, 0.89,
    0.84, ...]},
  {"qid": "Q67", "doc_ids": [...], "scores": [...]}
]
```


Only `doc_ids` are required for MAP. If `scores` are included, they must align with `doc_ids`. Each run entry contains the highest-ranked 10 document IDs, or all ranked documents when fewer than 10 candidates are available. MAP is computed from these stored results.

5.2 Non-Ranking Optimisation Requirement

Minimum requirement: implement and evaluate at least **one** *non-ranking* system optimisation. Submit `run_no_optimisation.json` using `method="no_optimisation"` (baseline) and `run_default.json` using `method="default"` (your submitted system, evaluated by hidden MAP). In the starter, `default` initially behaves the same as `no_optimisation`; after experimentation, `default` should represent the system you choose to submit.

Experiment runs: for attempted optimisations, create extra files under `runs/` with short descriptive method names.

Highest development/reporting band: include evidence from **two distinct optimisations** in addition to the baseline, with each optimisation isolated in a reproducible run. The optimisations must represent different system changes rather than repeated tuning of one setting. If you combine them, provide ablation evidence that distinguishes their individual and combined effects and supports analysis of their interaction. This requirement is for the final 1 mark of the Task 3 reporting component.

Examples include, but are not limited to, the following (choose one or more):

- Query expansion (e.g., synonyms applied *before* ranking),
- System-level document cleaning or query preprocessing, such as adapting your Part A cleaning ideas or removing query stop words,
- Candidate cutoffs or heuristic filtering *before* ranking.

Any additional optimisation run may use a descriptive filename under `runs/`.

Pipeline contract

You may modify implementation files used by the Task 3 pipeline, including `system/search_system.py`, or add helper files. The optimisation must remain non-ranking. Do not modify `test_sanity/`, `metrics/`, the provided data, or `AGENTS.md`. Keep the CLI arguments, required function signatures, and output JSON schema unchanged. During marking, your system will receive different documents and queries. Your method must generalise to unseen data rather than hard-code behaviour for specific development examples or IDs. After changing the pipeline, run `python -m test_sanity.check_submission`. Query preprocessing must preserve Task 1 syntax. For example, `AND` must not become `and`.

5.3 Dev-Set MAP Evaluation

Compute dev MAP over all runs in `./runs/` using:

```
python -m metrics.eval_map
```

5.4 Write-up requirements for Task 3

Optimisation description. Describe each optimisation. State where it acts in the retrieval pipeline, what behaviour it changes, and its assumptions. For each run, identify which changes are isolated or combined.

Results. Include a compact table comparing at least `no_optimisation` and `default`, with a dev MAP result for each included run. Include additional runs where they support the analysis or final choice. Every MAP value must be reproducible using `python -m metrics.eval_map`. Each main claim must be traceable to a run file under `runs/`.

Analysis. Explain how and why each discussed optimisation changed retrieval compared with `no_optimisation`. For each optimisation used to support the final judgement, analyse at least one concrete dev query. Trace how the optimisation's information retrieval mechanism changed relevant-document positions. Then explain the MAP difference in terms of how MAP is calculated from those rankings. Discuss where each optimisation helped or failed, account for interactions, and use run evidence to justify later decisions and the system selected as `default`. If `default` uses the same behaviour as `no_optimisation`, state this clearly and explain why the attempted optimisation was not selected.

6 One-Page Write-up

Submit `WRITEUP.pdf` using **11pt** text. Only the first page is marked. One page is a limit, not a target, and a concise report is valid. We assess the **completeness, validity, and insight** of your experimental reasoning rather than prose style or length. All reported results must be reproducible from the submitted code using the provided evaluation commands.

Organise the page under two clear headings: **Task 2: Ranking** and **Task 3: System Optimisation**. Under each heading, include a method or optimisation description, a results table, and analysis.

6.1 Generative AI Use Declaration

The one-page limit applies only to the Task 2 and Task 3 report content. Any required Generative AI declaration is excluded from this limit and is not marked. If you use Generative AI under the permitted Condition 2 rules, append the following declaration after your report. If you do not use Generative AI, no declaration is required.

"I used [tool/model] only for Condition 2 surface polish of [section(s)] in `WRITEUP.pdf`. I used it to [purpose]. I reviewed and revised the edits. No technical ideas, code, analysis, evidence, results, conclusions, report structure, or demonstration content were generated or changed."

If you use Generative AI, retain your original student-written draft and exact prompts until the assessment results are final. The draft may be rough or informally written, but it must already contain your technical content and reasoning. Do not submit these materials unless requested. We may select submissions for audit and ask you to provide them.

7 Recorded CLI Demonstration

Submit one continuous, unedited screen recording of no more than **3 minutes**. From the repository root, type and run `python -m test_sanity.check_submission`, then run either the `default` Pearson command or the MAP command from Quick Start. Choose one result visible in that output. Open a relevant code block and explain how the information retrieval principle implemented there contributed to the result.

Commands and navigation must be recorded live. Cuts, splicing, speed changes, or replaced audio result in 0. Brief pauses and corrections are acceptable. No face camera or slides are required. Production quality, accent, speaking speed, and typing speed are not assessed.

Use an accepted filename from the submission layout. Place the recording at the repository root and include it in the **same Canvas submission ZIP** as your code, runs, and `WRITEUP.pdf`. External video links are not marked.

8 Overall Code Quality Standards

Code quality is assessed holistically across the submitted code base (2 points). Use readable structure, meaningful names, concise documentation, and reusable logic. Handle invalid or unexpected inputs sensibly. Preserve required interfaces, layout, and the policy file. Use repository-relative file paths. Do not hard-code machine-specific paths or modify `sys.path`. Keep results deterministic, and remove unused imports and dead code.

Because Part B builds on Part A, meaningfully reuse or adapt your own Part A work in at least one relevant implementation. Merely retaining unused starter utility files does not satisfy this requirement.

9 Submission Rules and Checklist

Submission: Submit one completed assignment ZIP on Canvas. Package your code, runs, WRITEUP.pdf, and recorded demonstration together in this ZIP. The Canvas ZIP is the only source used for marking.

9.1 Submission Layout

Keep the complete starter folder structure. Your ZIP must extract to a repository root containing at least these paths:

```
index/
  builders.py
  access.py
  io.py
query_processing/
  detection.py
  query_process.py
  boolean.py
  wildcard.py
  proximity.py
ranking/
  rankers.py
system/
  search_system.py
metrics/
  eval_map.py
  eval_pear.py
runs/
  run_no_optimisation.json
  run_default.json
  run_<method_name>.json  # optional: extra optimisation evidence
data/
  dev/
  dev_ranking/
  glove/
utils/  # retain the utility files you submit
test_sanity/
  check_submission.py
requirements.txt
AGENTS.md
README.md
WRITEUP.pdf
DEMONSTRATION.mp4  # DEMONSTRATION.mov is also accepted
```

ZIP naming (please follow): Name your submission ZIP as YourName_s123456.zip (e.g., ZhuangLi_s123456.zip). This helps us match submissions quickly.

9.2 Repository Rules

- You may add helper functions and additional helper files as needed; however, **do not rename, move, or remove any required files, and do not change any required function signatures** in the starter code.
- Use only permitted libraries: `numpy`, `nlTK`, `bs4`, and the Python 3.10–3.14 standard library. Packages such as `scikit-learn`, `scipy`, `gensim`, and `pandas` are not permitted.
- Ensure there are **no code syntax errors** and **no missing imports** (e.g., importing non-permitted or unavailable libraries).
- Do not include virtual environments such as `.venv/`, `env/`, `venv/`, or `.conda/`.
- For marking, we apply a **5-minute** wall-clock **safety cutoff**. To sanity-check runtime on your machine, time an end-to-end dev run (Section 9.3); submissions that exceed the cutoff may be terminated and may not be marked for the affected task(s).
- `python -m test_sanity.check_submission` mimics key parts of the grading environment. It checks common import, interface, public-case, output-schema, and runtime problems, but passing it does not guarantee hidden-test correctness or full compliance with every submission rule.

9.3 Pre-submission checklist

Before you ZIP:

```
python -m test_sanity.check_submission
python -m metrics.eval_pear keyword_match
python -m metrics.eval_pear default
python -m system.search_system data/dev/queries.json data/dev/documents.
    jsonl runs/run_no_optimisation.json no_optimisation
python -m system.search_system data/dev/queries.json data/dev/documents.
    jsonl runs/run_default.json default
python -m metrics.eval_map
```

10 Academic Integrity and Prohibited Practices

Your submission must comply with the [RMIT academic integrity policy](#). Collaboration is limited to high-level discussion. Do not share code, pseudocode, or implementation details.

10.1 Generative AI Condition

This assessment uses **Condition 2: AI for surface polish**. Generative AI may only improve grammar, spelling, clarity, formatting, and language in text you have already written for `WRITEUP.pdf`. It must not generate or change ideas, code, algorithms, experiments, results, analysis, sources, conclusions, report structure, or demonstration content. See the Canvas [Generative AI Use in Assessment](#) page for the full Condition 2 guidance. This specification remains authoritative for this assessment. If you use Generative AI, include the declaration described in Section 6.1.

The repository policy file `AGENTS.md` states the same AI-assistance policy for coding tools; keep it in place.